

COM106: Introduction to Databases

SQL – More Operators and Join Queries

COM106 Weekly Schedule - Lecture & Lab Delivery

Week 9 Lab is the in-class exam (Set Exercise, 90-minute class test, 40% of module mark).

Week	Lecture	Lab / Seminar
1	Module Introduction & DB Basics and Security	Lab 1: Getting Started with MYSQL
2	Introduction to Entity-Relationship Modelling	Lab 2: Creating ER Diagrams with draw.io
3	ER Modelling 2 & Mapping to the Relational Model	Lab 3: ER Diagram Design from Scenarios
4	Normalisation	Lab 4: Normalisation Practice
5	Introduction to SQL (DDL, DML, DCL)	Lab 5: Building a Database with SQL (businessdb)
6	SQL Queries: Single-Table & Multiple-Table	Lab 6: SQL Retrieval Exercises (projemp)
7	**Further SQL: Set Operators & JOIN Queries** - UNION, INTERSECT & EXCEPT - LIKE wildcards & IS NULL - JOIN types & join conditions - INNER vs OUTER joins - Finding non-matching rows	**Lab 7: SQL Scripting (class_db & projemp)** - Creating class_db by script - Creating tables with data types - Dropping tables in FK order - Loading & verifying projemp

https://github.com/KodZone/COM106-MYSQL



Recap

- **SELECT Statement**: Retrieves data from one or more database tables using SELECT and FROM.
- **Filtering Records**: Use WHERE with operators such as AND, OR, NOT, IN and BETWEEN to select specific records.
- **Joining Tables**: Retrieve related data from multiple tables using WHERE joins (Mark 1) or INNER JOIN ... ON (Mark 2).
- **Aggregate Functions**: Use COUNT(), SUM(), AVG(), MIN() and MAX() to summarise data.
- **Grouping and Filtering Groups**: Use GROUP BY to organise records into groups and HAVING to filter groups based on aggregate conditions.
- **Sorting and Removing Duplicates**: Use ORDER BY to sort results and DISTINCT to eliminate duplicate rows.
- **Nested Queries (Subqueries)**: Use a SELECT statement inside another query to perform more complex data retrieval.
- **Views**: Use CREATE VIEW to define virtual tables that simplify queries and help control access to data.

LIKE Operator

LIKE Operator

SQL also provides a number of **Logical Operators** used to test for the truth of some condition. We have already seen AND, OR, BETWEEN, NOT and IN – (*there are a number of others*)

The LIKE Operator

LIKE allows the use of **wildcards** to perform **pattern matching** on an attribute in a query. It can be used in the WHERE clause of a SELECT, INSERT, UPDATE, or DELETE statement.

- **%** wildcard representing any string of characters
- **_** wildcard representing a single character
- **[ ]** looks for any match within a range of characters e.g. [a-m]
- **[^]** looks for any match not in the specified range e.g. [^a-m]

LIKE Operator

%	wildcard representing any string of characters
_	wildcard representing a single character
[ ]	looks for any match within a range of characters e.g. [a-m]
[^]	looks for any match not in the specified range e.g. [^a-m]

Some Examples:

Get the names of employees starting with 'S'

```
SELECT ename
FROM EMPLOYEE
WHERE ename LIKE 'S%';
```



ename
Smith
Smart

Get the names of employees containing the string 'row' anywhere

```
SELECT ename
FROM EMPLOYEE
WHERE ename LIKE '%row%';
```



ename
Brown
Thrower

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Smart	19000	3
852407	Brown	16000	3
852455	Bruce	25100	2
852491	Thrower	30500	1
852514	Dale	11650	2
852530	Dole	26980	4

LIKE Operator

%	wildcard representing any string of characters
_	wildcard representing a single character
[ ]	looks for any match within a range of characters e.g. [a-m]
[^]	looks for any match not in the specified range e.g. [^a-m]

Get the names of employees matching 'D?le'

```
SELECT ename
FROM EMPLOYEE
WHERE ename LIKE 'D_le';
```

ename
Dale
Dole

Get the names of employees with any characters from m to s

```
SELECT ename
FROM EMPLOYEE
WHERE ename LIKE '%[m-s]%';
```

ename
Smith
Smart
Brown
Bruce
Thrower

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Smart	19000	3
852407	Brown	16000	3
852455	Bruce	25100	2
852491	Thrower	30500	1
852514	Dale	11650	2
852530	Dole	26980	4

IS NULL Operator

IS NULL Operator

- The **IS NULL** operator in SQL is used to check whether a column contains a **NULL value**. It is important to understand that:
  - **NULL** is not a value but a state **indicating missing or unknown data**.
  - **NULL** is not the same as an empty string ('') or zero (0).
  - Standard comparison operators (such as = or !=) do not work with NULL.

- **IS NULL** used to check if a column contains a NULL value.
  - Returns TRUE when the column has NULL.

- **IS NOT NULL** used to check if a column contains a non-NULL value.
  - Returns TRUE when the column does not have NULL.

IS NULL Operator

Table: Employee

employee_id	first_name	last_name	department	email
1	Peter	Doe	Operations	peter@gmail.com
2	Megan	Morel	Finance	NULL
3	Rose	Bailey	Operations	rose@gmail.com
4	Linda	Bailey	Finance	NULL
5	Mary	Doe	Sales	NULL



```
SELECT *
FROM Employee
WHERE email IS NULL;
```



employee_id	first_name	last_name	department	email
2	Megan	Morel	Finance	NULL
4	Linda	Bailey	Finance	NULL
5	Mary	Doe	Sales	NULL

Set Operators (UNION, EXCEPT and INTERSECT)



Set Operators (UNION, EXCEPT and INTERSECT)

- In SQL, UNION, EXCEPT, and INTERSECT are known as Set Operations or Set Operators. These operators are used to combine, compare, or exclude results from multiple SELECT statements.

- **UNION** – Combines the results of two queries, removing duplicates.
  - **UNION ALL** includes duplicates.

- **INTERSECT** – Returns only the records that are present in both queries.

- **EXCEPT** – Returns records from the first query that do not appear in the second query.

UNION Operator

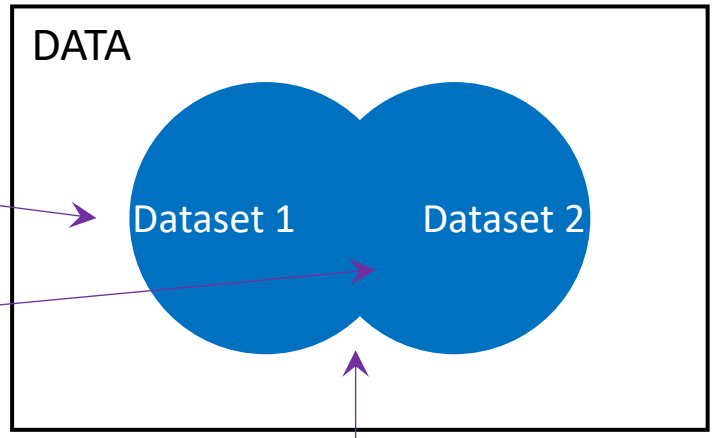
UNION - Combines two query statements (tables) across **compatible** attributes

- Requires the *same number of attributes* in each query
- Each corresponding pair of attributes must be *defined on the same domain* (compatible)
- Duplicate values are eliminated

```
SELECT <attribute_1>, …. < attribute_n>
FROM tables
[WHERE conditions]
UNION
SELECT <attribute_1>, …. < attribute_n>
FROM tables
[WHERE conditions];
```

Gives Dataset 1

Gives Dataset 2



The **UNION** operator selects only **distinct values** by default.

To allow duplicate values, use the ALL keyword - **UNION ALL**

*The **UNION** operator returns records in the shaded area - records that exist in Dataset 1 **or** Dataset 2*

UNION Operator

An Example of UNION

SUPPLIER

sup_no	s_name	country
s121	Big IT	Germany
s135	Top Design	Italy
s147	ALY Ltd	USA
s216	Microsoft	France
s227	Apple	Zambia
s251	PC Sport	Vietnam

CUSTOMER

cus_no	c_name	contact
c164	Apple	Jobs
c045	IT r us	Sinclair
c237	Microsoft	Gates
c256	Myco	Jones
c290	MFN Ltd	Smith
c515	HP	Bloggs

Get the names of all suppliers and customers

s_name and c_name must have the same datatype

```
SELECT s_name
FROM SUPPLIER
UNION
SELECT c_name
FROM CUSTOMER
```

All records in SUPPLIER and CUSTOMER

s_name
Big IT
Top Design
ALY Ltd
Microsoft
Apple
PC Sport
IT r us
Myco
MFN Ltd
HP

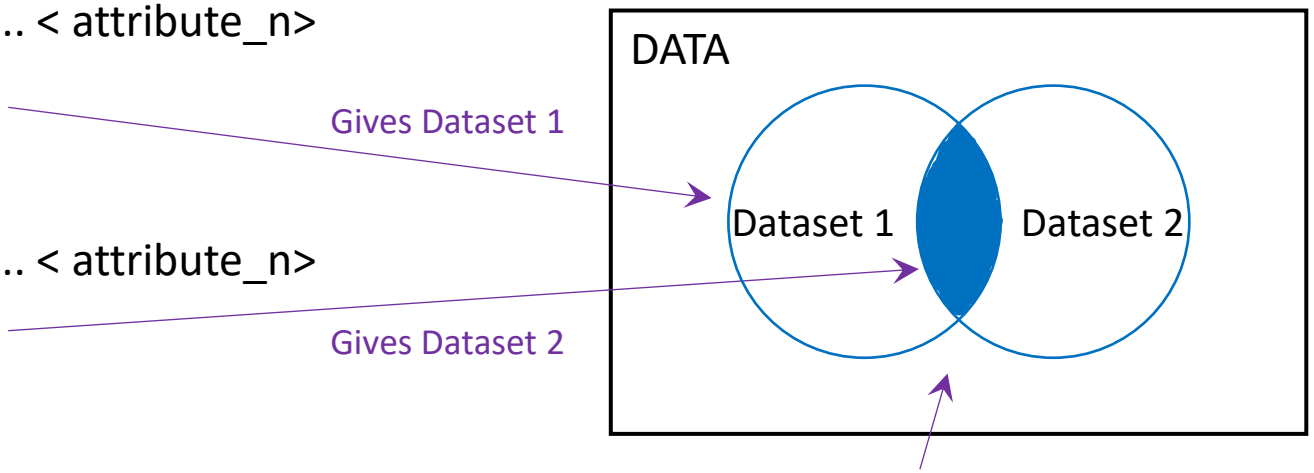
- Q – Can we use the **WHERE** Clause with **IN / NOT IN** besides to **AND / OR** operators in this case?

INSPECT Operator

INTERSECT - Returns values found in both queries

- Requires the *same number of attributes* in each query
- Each corresponding pair of attributes must be *defined on the same domain* (compatible)

```
SELECT <attribute_1>, …. < attribute_n>
FROM tables
[WHERE conditions]
INTERSECT
SELECT <attribute_1>, …. < attribute_n>
FROM tables
[WHERE conditions];
```



*The **INTERSECT** operator returns records in the shaded area - records that exist in **both** Dataset 1 **and** Dataset 2*

INSPECT Operator

An Example of INTERSECT

SUPPLIER

sup_no	s_name	country
s121	Big IT	Germany
s135	Top Design	Italy
s147	ALY Ltd	USA
s216	Microsoft	France
s227	Apple	Zambia
s251	PC Sport	Vietnam

CUSTOMER

cus_no	c_name	contact
c164	Apple	Jobs
c045	IT r us	Sinclair
c237	Microsoft	Gates
c256	Myco	Jones
c290	MFN Ltd	Smith
c515	HP	Bloggs

Get the names of suppliers that are also customers

s_name and c_name must have the same datatype

```
SELECT s_name
FROM SUPPLIER
INTERSECT
SELECT c_name
FROM CUSTOMER
```

s_name
Microsoft
Apple

***Microsoft** and **Apple** are in both SUPPLIER and CUSTOMER*

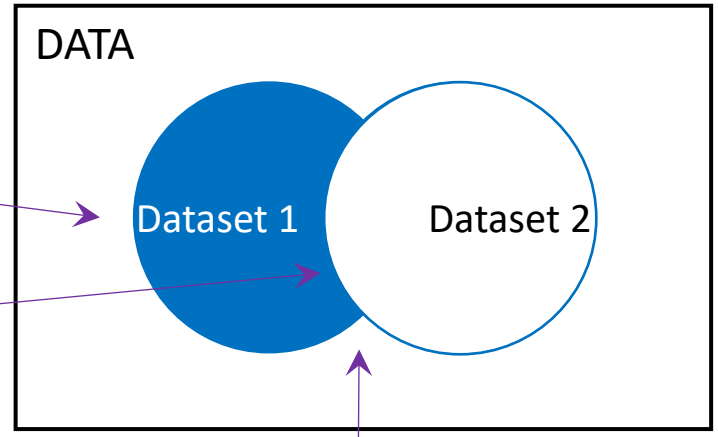
- Q – Can we use the **WHERE** Clause with **IN / NOT IN** besides to **AND / OR** operators in this case?

EXCEPT Operator

Except - Returns values found in the first query not found in second query

- Requires the *same number of attributes* in each query
- Each corresponding pair of attributes must be *defined on the same domain* (compatible)

```
SELECT <attribute_1>, …. < attribute_n>
FROM tables
[WHERE conditions]
EXCEPT
SELECT <attribute_1>, …. < attribute_n>
FROM tables
[WHERE conditions];
```



*The **EXCEPT** operator returns records in the shaded area - records that exist in Dataset 1 **and not in** Dataset 2*

EXCEPT Operator

An Example of EXCEPT

A company holds details of its **SUPPLIERS** and its **CUSTOMERS**

SUPPLIER

sup_no	s_name	country
s121	Big IT	Germany
s135	Top Design	Italy
s147	ALY Ltd	USA
s216	Microsoft	France
s227	Apple	Zambia
s251	PC Sport	Vietnam

CUSTOMER

cus_no	c_name	contact
c164	Apple	Jobs
c045	IT r us	Sinclair
c237	Microsoft	Gates
c256	Myco	Jones
c290	MFN Ltd	Smith
c515	HP	Bloggs

```
SELECT s_name
FROM SUPPLIER
EXCEPT
SELECT c_name
FROM CUSTOMER
```

s_name
Big IT
Top Design
ALY Ltd
PC Sport

Get the names of suppliers that are not also customers

s_name and c_name must have the same datatype

***Microsoft** and **Apple** from SUPPLIER are not in the results because they occur in CUSTOMER*

- Q – Can we use the **WHERE** Clause with **IN / NOT IN** besides to **AND / OR** operators in this case?

JOIN Operators



JOIN Operators

- SQL **JOIN** operators are used to combine rows from two or more tables based on a **related column**. They help retrieve data from multiple tables efficiently.

- **Types of SQL JOINs -** SQL provides different types of JOIN operators:
  1) **INNER JOIN** – Returns only matching rows from both tables.
  2) **LEFT JOIN (LEFT OUTER JOIN)** – Returns all rows from the left table and matching rows from the right table.
  3) **RIGHT JOIN (RIGHT OUTER JOIN)** – Returns all rows from the right table and matching rows from the left table.
  4) **FULL JOIN (FULL OUTER JOIN)** – Returns all rows from both tables, including unmatched ones.
  5) **CROSS JOIN** – Returns the Cartesian product of both tables (all possible row combinations).
  6) **SELF JOIN** – Joins a table with itself.

JOIN Operators

Mark 1 - *implicit join notation*

```
SELECT ename, role
FROM EMPLOYEE, WORKS_ON
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND floor = 1;
```

Mark 2 - *explicit join notation*

```
SELECT ename, role
FROM EMPLOYEE INNER JOIN WORKS_ON
ON EMPLOYEE.enum = WORKS_ON.enum
WHERE floor = 1;
```

The JOIN condition is specified in the FROM clause as:

FROM *table1* ***JOIN_TYPE*** *table2 [****ON*** *(****join_condition****)]*

The **join_condition** defines the way two tables are related in a query by specifying:

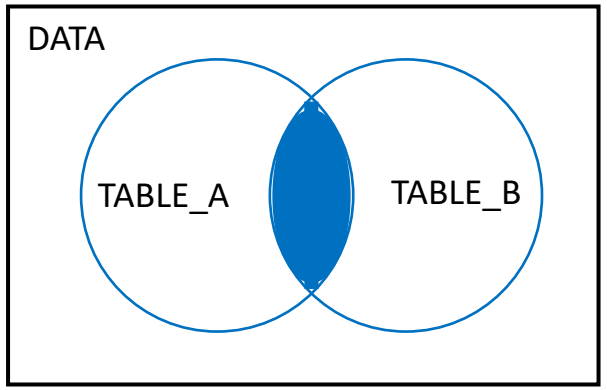
- The column from each table to be used for the join (typically, ***but not necessarily*****, on a PK/FK match**).
- A logical operator (for example, = or <>, etc) to be used in comparing values in the columns.

'=' is most commonly used – also known as an ***equi-join***

Explicit join notation is considered **best practice** since the join conditions are separated from any other conditions in the WHERE clause.

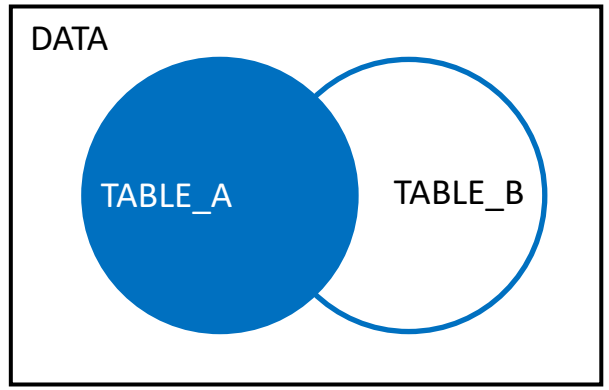
JOIN Operators

TABLE_A INNER JOIN TABLE_B



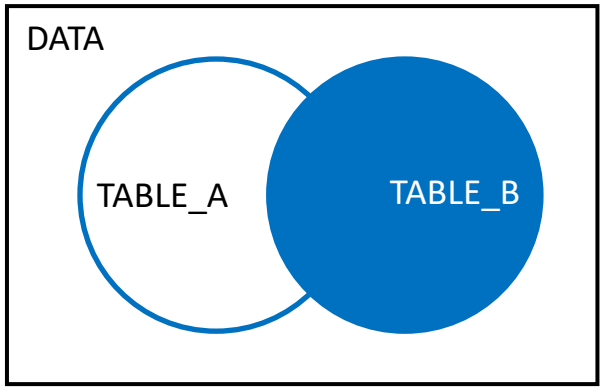
INNER JOIN - Select all records from TABLE_A and TABLE_B, where the join condition is met.

TABLE_A LEFT OUTER JOIN TABLE_B



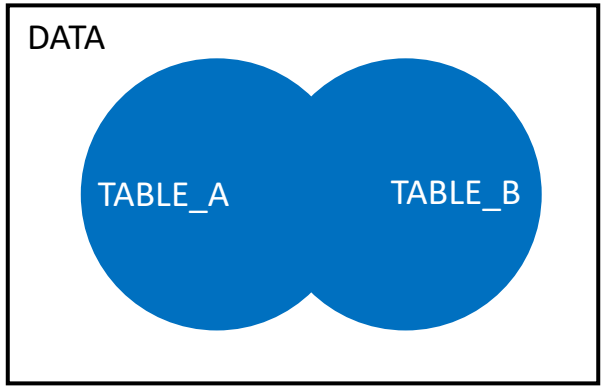
LEFT OUTER JOIN - Select all records from TABLE_A, along with records from TABLE_B for which the join condition is met (if at all).

TABLE_A RIGHT OUTER JOIN TABLE_B



RIGHT OUTER JOIN - Select all records from TABLE_B, along with records from TABLE_A for which the join condition is met (if at all).

TABLE_A FULL OUTER JOIN TABLE_B

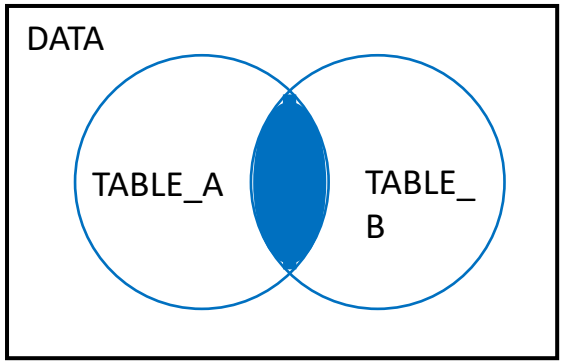


FULL OUTER JOIN - Select all records from TABLE_A and TABLE_B, regardless of whether the join condition is met or not.

INNER JOIN Operator

Consider the following schema:

CLUB (<u>club_id</u>, club_name)
MEMBER (<u>member_id</u>, member_name, club_id*)



CLUB

club_id	club_name
C1	Tennis
C2	Archery
C3	Judo

MEMBER

member_id	member_ name	club_id
M20	Smith	C3
M21	Jones	*Null*
M22	White	C1
M23	Black	C3
M24	Green	C1
M25	Brown	*Null*

INNER JOIN

```
SELECT *
FROM MEMBER INNER JOIN CLUB
ON MEMBER.club_id = CLUB.club_id;
```

Note: Members “Jones" and “Brown” and the “Archery” club do not appear in the results. Neither of these has any matching rows in the other respective table.

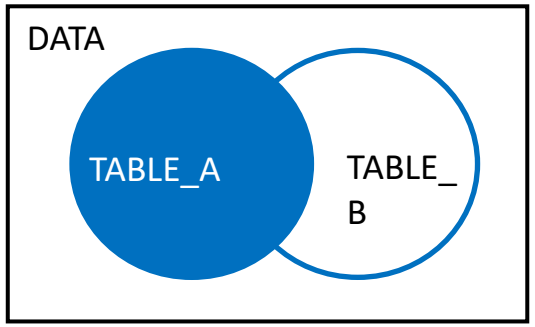
RESULT TABLE

member_id	member_ name	club_id	club_name
M20	Smith	C3	Judo
M22	White	C1	Tennis
M23	Black	C3	Judo
M24	Green	C1	Tennis

LEFT OUTER JOIN Operator

Consider the following schema:

CLUB (club_id, club_name)
MEMBER (member_id, member_name, club_id*)



LEFT OUTER JOIN

LEFT Table *RIGHT Table*

```
SELECT *
FROM MEMBER LEFT OUTER JOIN CLUB
ON MEMBER.club_id = CLUB.club_id;
```

Note: A **LEFT JOIN** returns all the values from the left table, plus matched values from the right table or *NULL* in case of no matching join predicate.

CLUB

club_id	club_name
C1	Tennis
C2	Archery
C3	Judo

MEMBER

member_id	member_name	club_id
M20	Smith	C3
M21	Jones	*Null*
M22	White	C1
M23	Black	C3
M24	Green	C1
M25	Brown	*Null*

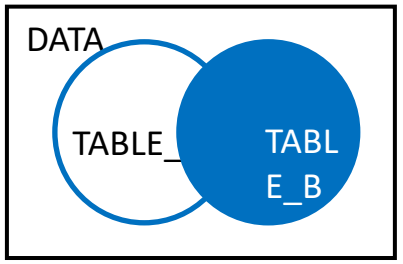
RESULT TABLE

member_id	member_name	club_id	club_name
M20	Smith	**C3**	Judo
M21	Jones	***Null***	*Null*
M22	White	**C1**	Tennis
M23	Black	**C3**	Judo
M24	Green	**C1**	Tennis
M25	Brown	***Null***	*Null*

RIGHT OUTER JOIN Operator

Consider the following schema:

CLUB (club_id, club_name)
MEMBER (member_id, member_name, club_id*)



RIGHT OUTER JOIN

```
SELECT *
FROM MEMBER RIGHT OUTER JOIN CLUB
ON MEMBER.club_id = CLUB.club_id;
```

Note: A **RIGHT JOIN** is similar to a LEFT JOIN, but with the treatment of the tables reversed. Returns all the values from the right table, plus matched values from the left table or *NULL* in case of no matching join predicate.

Notice- MEMBER **RIGHT OUTER JOIN** CLUB gives the same result as CLUB **LEFT OUTER JOIN** MEMBER

CLUB

club_id	club_name
C1	Tennis
C2	Archery
C3	Judo

MEMBER

member_id	member_ name	club_id
M20	Smith	C3
M21	Jones	*Null*
M22	White	C1
M23	Black	C3
M24	Green	C1
M25	Brown	*Null*

RESULT TABLE

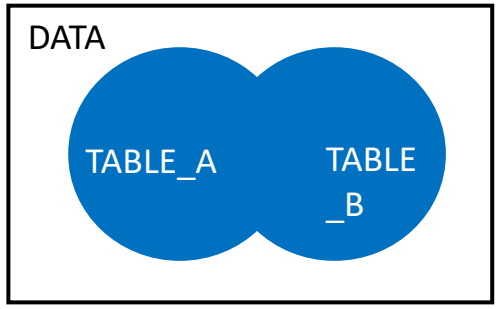
member_id	member_ name	club_id	club_name
M20	Smith	C3	Judo
M22	White	C1	Tennis
M23	Black	C3	Judo
M24	Green	C1	Tennis
Null	*Null*	C2	Archery

The order of the tables is important

FULL OUTER JOIN Operator

Consider the following schema:

CLUB (club_id, club_name)
MEMBER (member_id, member_name, club_id*)



CLUB

club_id	club_name
C1	Tennis
C2	Archery
C3	Judo

MEMBER

member_id	member_name	club_id
M20	Smith	C3
M21	Jones	*Null*
M22	White	C1
M23	Black	C3
M24	Green	C1
M25	Brown	*Null*

FULL OUTER JOIN

```
SELECT *
FROM MEMBER FULL OUTER JOIN CLUB
ON MEMBER.club_id = CLUB.club_id;
```

Note: A **FULL JOIN** combines the results of both left and right outer joins.

The joined table will contain all records from both tables, with *NULL*s for missing matches on either side.

Some RDBMS do not support FULL JOINs, eg., MySQL

RESULT TABLE

member_id	member_name	club_id	club_name
M20	Smith	**C3**	Judo
M21	Jones	***Null***	*Null*
M22	White	**C1**	Tennis
M23	Black	**C3**	Judo
M24	Green	**C1**	Tennis
M25	Brown	***Null***	*Null*
Null	*Null*	**C2**	Archery

Self Joining

Self Joins

In some cases a table needs to be **joined to itself**, particularly if extra information is stored in the table but not explicitly available by a normal query.

e.g. ***Find all pairs of employees who work on the same floor as each other.***

```
SELECT EMPLOYEE.ename, TEMP.ename
FROM EMPLOYEE, EMPLOYEE AS TEMP
WHERE EMPLOYEE.floor = TEMP.floor
AND EMPLOYEE.enum < TEMP.enum ;
```

This is an example of a **SELF JOIN** whereby a table is joined to itself.

Result Table

ename	ename
Smith	Adams
Jones	Brown
White	Doyle

To do this one copy of the table is given the **alias TEMP** (signified as **TABLENAME AS ALIAS**)

Joins are performed where the floor numbers are the same:

EMPLOYEE.floor = TEMP.floor

the clause:

EMPLOYEE.enum < TEMP.enum

eliminates an employee being shown as on the **same floor as himself** and **duplicate pairs** (e.g. either Adams, Smith OR Smith, Adams) are eliminated

EMPLOYEE

enum	ename	floor
852341	Smith	*1*
852358	Jones	*3*
852407	Brown	3
852455	White	*2*
852491	Adams	1
852514	Doyle	2
852530	Evans	4

TEMP

floor	enum	ename
1	852341	Smith
3	852358	Jones
3	852407	Brown
2	852455	White
1	852491	Adams
2	852514	Doyle
4	852530	Evans



Summary

- **LIKE Operator**: Performs pattern matching using wildcards such as % (multiple characters) and _ (single character).
- **NULL Handling**: Use IS NULL and IS NOT NULL to identify missing or unknown values.
- **Set Operators**: Use UNION to combine results, INTERSECT to find common records, and EXCEPT to identify records present only in the first query.
- **INNER JOIN**: Retrieves only records that satisfy the matching condition between tables.
- **LEFT and RIGHT OUTER JOIN**: Retrieve all records from one table, including unmatched records, with NULL values where no match exists.
- **FULL OUTER JOIN**: Retrieves all records from both tables, including unmatched records from either side.
- **Finding Unmatched Records**: Combine OUTER JOIN with IS NULL to identify records without corresponding matches.
- **SELF JOIN**: Joins a table with itself using aliases to compare related records within the same table.




THE
END